Multimodal Stock Trend Predictor: Architecture & Roadmap

1. Validating Your Approach & Necessary Modifications

Your overall intuition is exactly right: `[Price] + [News] + [Macro] -> [Fusion] -> [Prediction]`. However, to make this work in practice, we need to make three critical modifications:

A. The "Four Models" Training Strategy

You mentioned training four models. In modern deep learning, we don't usually train them entirely separately.

- **The News Model (FinBERT):** We will use this as a **frozen feature extractor**. You will pre-process your 10 years of text through FinBERT once and save the embeddings. Training FinBERT simultaneously with price data is computationally too expensive and prone to overfitting.
- **The Price (TCN), Macro (MLP), and Fusion Models:** These will be constructed as **one single unified neural network** that is trained end-to-end. The TCN learns to extract price features specifically to complement the macro and news data at the fusion layer.

B. Price Data Noise (The 1-Minute Problem)

Indian stock market data at the 1-minute level (375 bars/day) is incredibly noisy due to bid-ask bounce and micro-volatility.

- **Modification:** Instead of *only* taking the "log price," you should feed the model `[Log Return, Log Volume, High-Low Spread]`.
- **Model Selection: TCN (Temporal Convolutional Network)** is 100% the right choice here. An LSTM will forget early data in a 3,750-step sequence (10 days * 375 bars). A TCN uses dilated convolutions to handle massive sequences flawlessly and trains much faster.

C. The Reality of Results (Expectation Setting)

Can you expect good results? Yes, but define "good" carefully. In financial ML, the signal-to-noise ratio is extremely low.

- A model predicting a 5-day forward trend with **54% to 58% accuracy** is considered exceptionally good and highly profitable in algorithmic trading. Do not expect 80%+ accuracy; if you see that, the model has a data leakage bug.

2. Model Finalization

Here is the finalized model architecture we will build:

1. **Price Branch: TCN (Temporal Convolutional Network).** Input shape: `(Batch, TimeSteps, Features)`. Output: A 128-dimensional embedding vector.
2. **News Branch: FinBERT (Pre-trained) + Temporal Attention Layer.** FinBERT creates 768-dimensional vectors for each article. The Temporal Attention layer learns which news from the last 10 days is most relevant today. Output: A 64-dimensional embedding vector.
3. **Macro Branch: Multi-Layer Perceptron (MLP).** Simple dense layers with dropout. Output: A 32-dimensional embedding vector.

4. **Fusion & Prediction Head: Concatenation + Dense Layers.** We concatenate `[128 + 64 + 32] = 224` dimensions, pass it through a Dense network, and output a Softmax layer for 3 classes: `[UP, DOWN, FLAT]`.

3. The 4-Phase Execution Plan

To execute this step-by-step, we will follow this roadmap.

Phase 1: Data Acquisition & Preprocessing (The Hardest Part)

- **Goal:** Source the data, clean it, and structure it into the overlapping sliding windows.
- **The News Challenge:** Getting 10 years of historical, stock-specific Indian financial news is difficult. We will need to discuss sources (e.g., scraping Moneycontrol/Economic Times, using GDELT, or APIs like Tiingo) and how to map articles to specific NSE/BSE tickers using Named Entity Recognition (NER).
- **Output of this phase:** Three synchronized Numpy arrays/Tensors for `X_price`, `X_news`, `X_macro`, and the target array `Y_trend`.

Phase 2: Building the Price Branch (TCN)

- **Goal:** Architect the Temporal Convolutional Network using PyTorch or TensorFlow.
- **Task:** Code the dilated causal convolutions, residual blocks, and test it independently on the price data to ensure the loss decreases.

Phase 3: Building the News & Macro Branches

- **Goal:** Set up the text pipeline and macro processor.
- **Task:** Write the script that converts raw text into FinBERT embeddings. Build the Temporal Attention layer for the news branch, and the MLP for the macro branch.

Phase 4: Fusion, Training, and Evaluation

- **Goal:** Connect all branches into the final multimodal network.
- **Task:** Write the custom training loop. Implement strict time-series cross-validation (Purged K-Fold or Walk-Forward validation) to ensure zero data leakage. Evaluate the model using precision, recall, and Sharpe Ratio.